Encryption in the microarchitectural world

Ping-Lun Wang, Fraser Brown, Riccardo Paccagnella,
Eyal Ronen, Riad S. Wahby, and Yuval Yarom

RWC, 03/09/2026

Carnegie Mellon University CyLab Security and Privacy Institute

TEL AVIV UNIVERSITY אוניברסיטת תל אביב

RUHR UNIVERSITÄT BOCHUM RUB

Arch vs. µArch components

▲ CPU

- Cache hierarchy
- Branch predictor (BP)
- …

Microarchitectural (µArch) components

- Instruction set
- Architectural registers
- …

Architectural (Arch) components

Credit: irasutoya.com

µWMs: hidden computers in the CPU

µArch weird machines (µWMs)

▲ CPU

µWMs: difficult to analyze!

😎 Hides computational states

🤯 Prevents code analysis

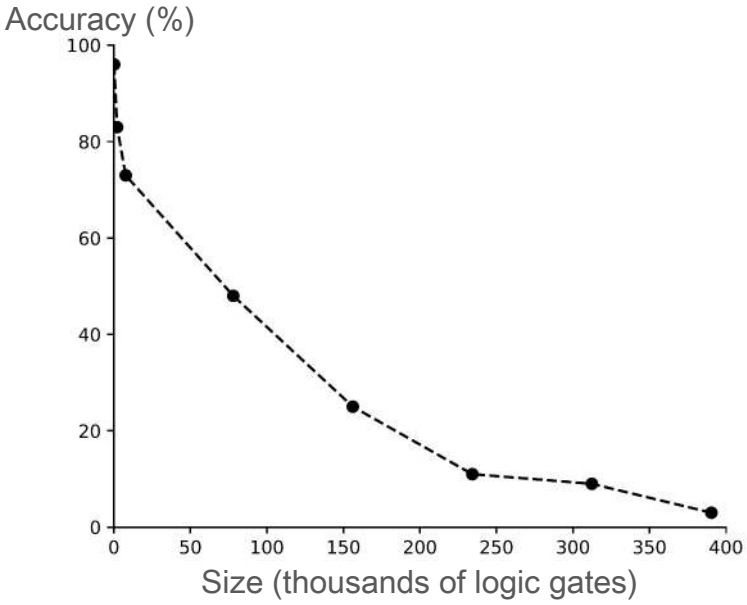
😈 Defeats debuggers and emulators

🤔 Seems to be great for program obfuscation?

Credit: irasutoya.com

Existing µWMs need improvements…

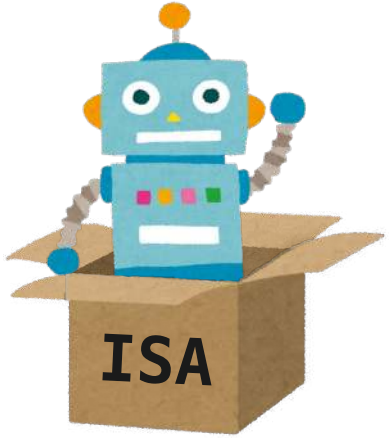
- Not scalable for large computations



- Difficult to create new µWMs

```
_32BitValueToCacheState(C_arch, C1, NULL);
_32BitValueToCacheState(D_arch, D1, NULL);
_32BitValueToCacheState(E_arch, E, NULL);
_32BitValueToCacheState(K[0], K0, NULL);
_32BitValueToCacheState(W_arch, W, NULL);
LeftShift(A1, A1, 27);
mfence();
lfence();
trash = _32BitAnd(B1, C1, B_AND_C, trash);
mfence();
lfence();
trash = _32BitNot(B2, NOT_B, trash);
mfence();
lfence();
trash = _32BitAnd(NOT_B, D1, NOT_B_AND_D,
trash);
… // 128 lines in total
```

Our work: Flexo and Transient Arch Execution



Flexo

- ✨ Improved construction!
- ✨ Compiler for μWMs!
- ✨ μWM binary packer!

Bending microarchitectural weird machines towards practicality [USENIX Security 24]

Transient Arch Execution

- ✨ New μWM: weird programs!
- ✨ Large state storage!
- ✨ 543× speedup!

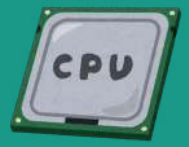
Transient architectural execution: from weird gates to weird programs [SP 26]

Credit: irasutoya.com

Background

µWMs and weird computation

From Arch to µArch “weird” computation



Architectural Computation

- ✅ **State: registers and memory**
  Architecturally visible
- ✅ **Compute: ISA instructions**
  ADD, AND, MOV, JMP

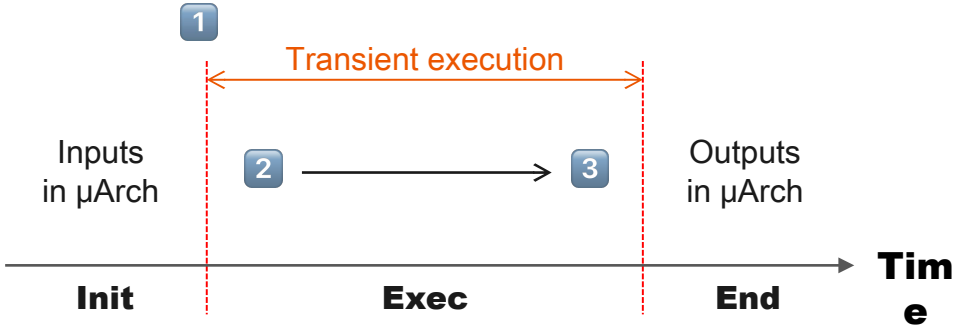
Microarchitectural “Weird” Computation

- 🔥 **State: “weird” registers using µArch state**
  Cached = 1
  Not cached = 0
- 🔥 **Compute: hardware side-effects**
  “Weird gates” compute with memory operations during transient execution

Credit: irasutoya.com

The construction of a weird gate

How weird gates work	
1	**Trigger transient execution**
2	**Compute with input latency**
3	**Access output weird registers**



The construction of a weird gate

How weird gates work

1. **Trigger transient execution**
   Branch misprediction, return address misprediction, etc.

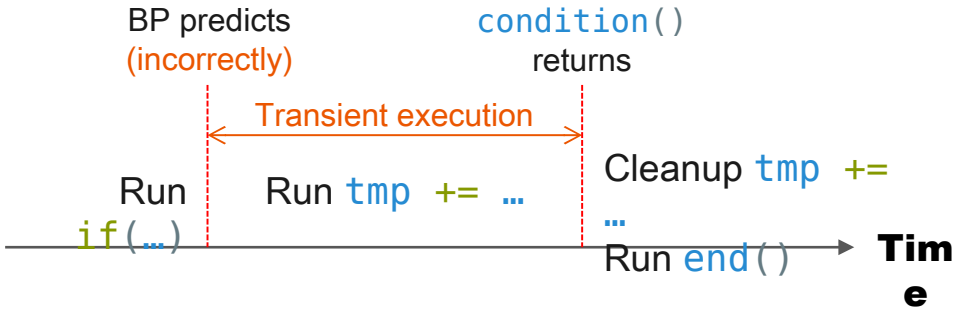
2. **Compute with input latency**
   Depending on the cache state, input weird registers may have different latency.
   Cached → short cache hit
   Not cached → long cache miss

3. **Access output weird registers**
   Write the results to output weird registers via memory loads.

Slow…

```
if (condition()) {
    tmp += out[in1[0] +
in2[0]];
}
end();
```

Misprediction



The construction of a weird gate

How weird gates work

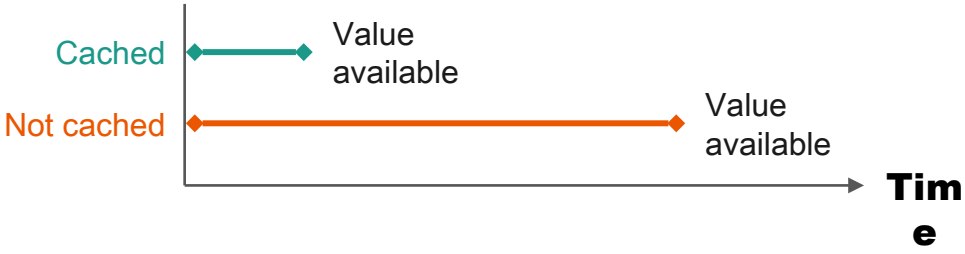
1. **Trigger transient execution**
   Branch misprediction, return address misprediction, etc.

2. **Compute with input latency**
   Depending on the cache state, input weird registers may have different latency.
   Cached → short cache hit
   Not cached → long cache miss

3. **Access output weird registers**
   Write the results to output weird registers via memory loads.

```
if (condition())
    tmp += out[in1[0] +
in2[0]];
```

Input weird registers



The construction of a weird gate

How weird gates work

1. **Trigger transient execution**
   Branch misprediction, return address misprediction, etc.

2. **Compute with input latency**
   Depending on the cache state, input weird registers may have different latency.
   Cached → short cache hit
   Not cached → long cache miss

3. **Access output weird registers**
   Write the results to output weird registers via memory loads.

```
if (condition())
    tmp += out[in1[0] +
in2[0]];
```

Output weird register

- Initial state: not cached
- After execution
  - Result is 0: not cached
  - Result is 1: cached

Prior work's AND gate

```
if (condition()) tmp += out[in1[0] + in2[0]];
```

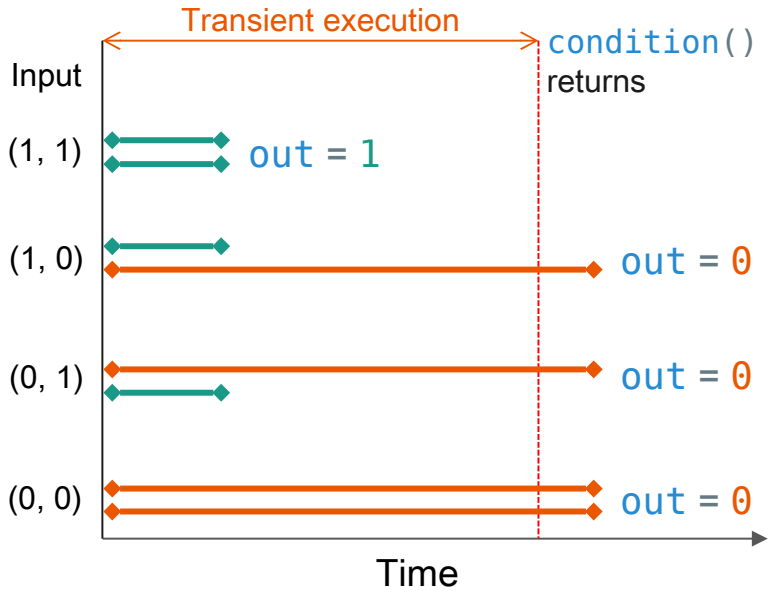
Misprediction

How the AND gate works

1. **Trigger transient execution**
   CPU mispredicts the branch and executes the gate transiently.

2. **Compute with input latency**
   - Both `in1` and `in2` are fetched at the same time (no dependency!)
   - Addition (`in1[0] + in2[0]`) is computed after the loads

3. **Access output weird registers**
   `out` is fetched **only when** 2 finished before transient execution ends.



Prior work's other types of gates

```
if (condition()) {
    tmp +=
out[in1[0]];
    tmp +=
out[in2[0]];
}
```

▲ OR gate

```
if (in[0])
    tmp +=
out[delay()];
```

▲ NOT gate

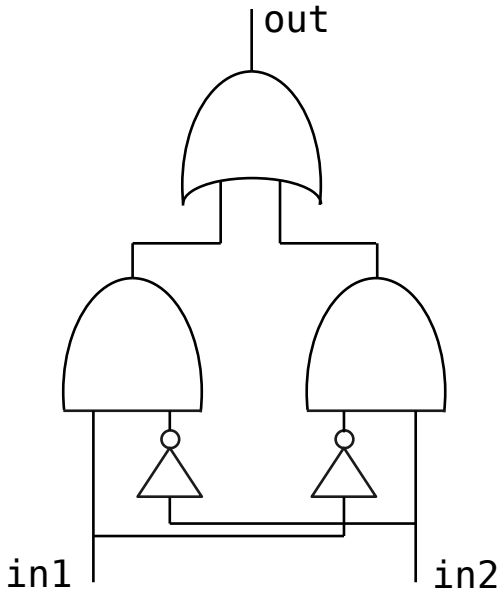
```
if (condition()) {
    tmp += out[in1[0]];
    tmp += out[in2[0] +
in3[0]];
}
```

OR

AND

▲ AND & OR gate

Compose weird gates into weird circuits



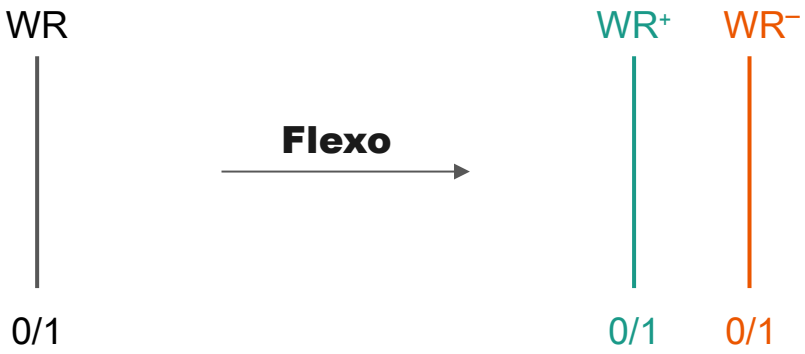
▲ Prior work's XOR circuit (5 weird gates)

✨ **Flexo can do this in one gate!!**

Flexo

Making weird gates much faster and much more accurate

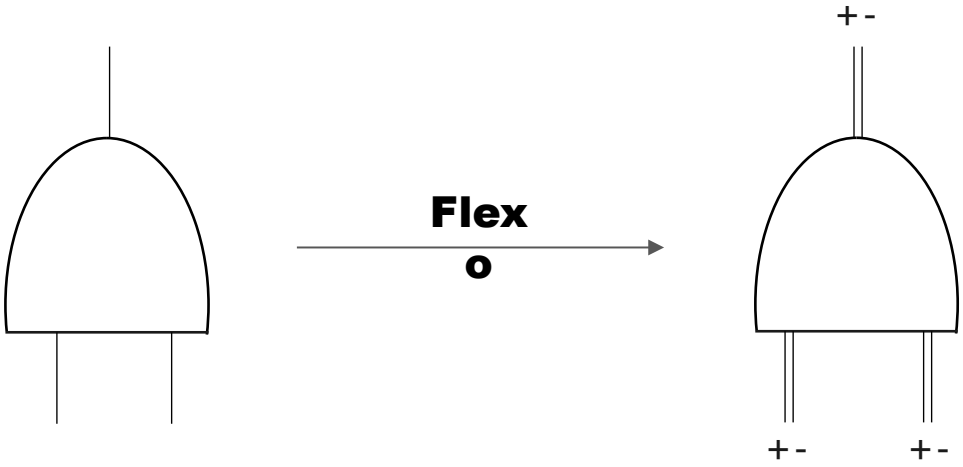
Flexo's construction: differential encoding



WR^+	WR^-	Value
0	1	Binary 0
1	0	Binary 1

WR^+	WR^-	**Value**
0	**1**	Binary 0
1	0	Binary 1
0	0	**Invalid!**
1	**1**	**Invalid!**

Differential encoding → many benefits!



✨ Powerful gates!

✨ Higher accuracy!

Applying Flexo's differential encoding

in1	in2	out
1	0	1
0	1	1
1	1	0
0	0	0

→ `out` is 1 **when** `in1` is 1 **AND** `in2` is 0

→ $\texttt{out} = \texttt{in1} \wedge \neg\texttt{in2}$

Flexo

→ $\texttt{out}^{+} = \texttt{in1}^{+} \wedge \texttt{in2}^{-}$

Applying Flexo's differential encoding

in1	in2	out
1	0	1
0	1	1
1	1	0
0	0	0

$\rightarrow$ $out^+ = in1^+ \wedge in2^-$

$\rightarrow$ $out^+ = in1^- \wedge in2^+$

$\rightarrow$ $out^- = in1^+ \wedge in2^+$

$\rightarrow$ $out^- = in1^- \wedge in2^-$

OR them together $\rightarrow$

$$out^+ = (in1^+ \wedge in2^-) \vee (in1^- \wedge in2^+)$$
$$out^- = (in1^+ \wedge in2^+) \vee (in1^- \wedge in2^-)$$

✨ **Only AND & OR operations!!**

New encoding → much more scalable gates!

in1	in2	out
1	0	1
0	1	1
1	1	0
0	0	0

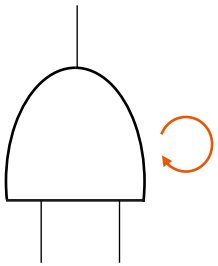
```
if (condition()) {
    tmp += out_p[in1_p[0] +
in2_m[0]];
    tmp += out_p[in1_m[0] +
in2_p[0]];
    tmp += out_m[in1_p[0] +
in2_p[0]];
    tmp += out_m[in1_m[0] +
in2_m[0]];
}
```

OR operators

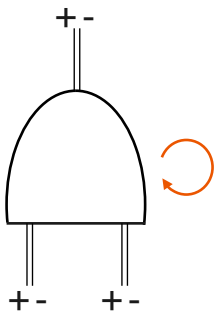
AND operators

✨ **Any gate with ≤ 4 inputs works!!**
E.g., 4-bit XOR or 2-bit adder

Error correction with dynamic voting



Run 1: 1
Run 2: 1
Run 3: 1
Run 4: 1
Run 5: 0
→ Output 1



Run 1: Invalid!
Run 2: Invalid!
Run 3: 1
→ Output 1

Prior work (GoT[1]): 3-out-of-5 voting

Flexo: rerun only when error occurs

✨ **Higher accuracy & less overhead!!**

[1] The Gates of Time: Improving Cache Attacks with Transient Execution, Katzman et al., USENIX Security ‘23

Flexo's compiler: the first compiler for

```
_32BitValueToCacheState(C_arch, C1, NULL);
_32BitValueToCacheState(D_arch, D1, NULL);
_32BitValueToCacheState(E_arch, E, NULL);
_32BitValueToCacheState(K[0], K0, NULL);
_32BitValueToCacheState(W_arch, W, NULL);
LeftShift(A1, A1, 27);
mfence();
lfence();
trash = _32BitAnd(B1, C1, B_AND_C, trash);
mfence();
lfence();
trash = _32BitNot(B2, NOT_B, trash);
mfence();
lfence();
trash = _32BitAnd(NOT_B, D1, NOT_B_AND_D, trash);
mfence();
lfence();
trash = _32BitOr(B_AND_C, NOT_B_AND_D,
B_AND_C_OR_NOT_B_AND_D, trash);
mfence();
lfence();
trash = _32BitAdder_impl(A1,
B_AND_C_OR_NOT_B_AND_D, SUM_1, WORDSIZE, trash);
mfence();
lfence();
...
```

(128 lines in total)

→

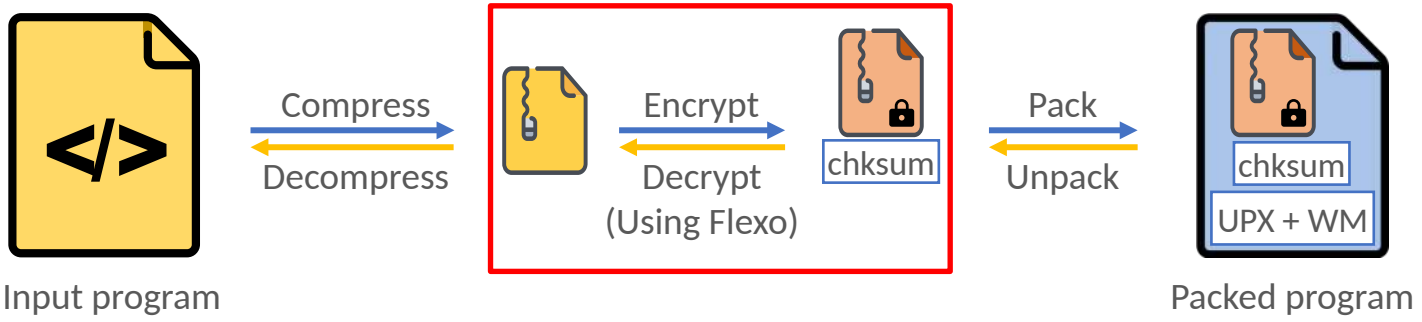
```
void __weird__sha1_round(
    unsigned* input,
    unsigned* output, unsigned* error_output
) {
    unsigned a = input[0];
    unsigned b = input[1];
    unsigned c = input[2];
    unsigned d = input[3];
    unsigned e = input[4];
    unsigned w = input[5];
    unsigned k = 0x5A827999;

    unsigned f = (b & c) | ((~b) & d);
    unsigned temp = ROL(a, 5) + f + e + w +
k;

    output[0] = temp;
    output[1] = a;
    output[2] = ROL(b, 30)
    output[3] = c;
    output[4] = d;
}
```

(21 lines in total)

😇 No low-level details

✨ Optimizations!

UPX + Flexo: the first µWM binary packer



Roughly 1 minute to unpack 132 KB using Simon
(5 minutes using AES)

➡ **Slower, but defeats anti-virus tools!**

Limitations of weird gates

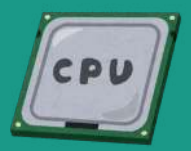
- Branches and memory operations are not scalable
- A weird register can only store 1 bit
- A weird gate can only compute 4 bits at a time

🤔 Can we make µWMs more powerful?

Transient Arch Exec

From weird gates to weird programs

Transient arch exec: µArch state + Arch compute



Architectural Computation

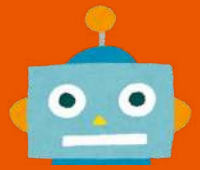
✅ **State: registers and memory**
Architecturally visible

✅ **Compute: ISA instructions**
ADD, AND, MOV, JMP

2 Arch compute

1 µArch to Arch

3 Arch to µArch



Microarchitectural "Weird" Computation

🔥 **State: "weird" registers using µArch state**
Cached = 1
Not cached = 0

🔥 **Compute: hardware side-effects**
"Weird gates" compute with transient memory operations

Plan: run 1 2 3 in one transient execution window

Credit: irasutoya.com

Convert cache states to Arch states

```
int cache2arch(int
*input) {
    if (input[0]) return
0;
    else return 1;
}
```

- `input[0]` is set to 0 architecturally
- Return value (during transient execution):
  - `input` is cached: return 1
  - `input` is not cached: return 0

Convert cache states to Arch states

```
int cache2arch(int
*input) {
    if (input[0]) return
0;
    else return 1;
}
```

0

Cached

```
int cache2arch(int
*input) {
    if (input[0]) return
0;
    else return 1;
}
```

??? Misprediction

Not cached

Weird functions: new computational abilities

```
if (condition()) {
    int tmp_in1 = cache2arch(in1);
    int tmp_in2 = cache2arch(in2);
    if (tmp_in1 ^
tmp_in2) load(out);
}
```

- ✨ Powerful instructions (e.g., AES-NI)!
- ✨ Branch instructions!
- ✨ Dynamically access weird registers!
- ✨ Access to the (read-only) main memory!
- ✨ Can be composed into weird programs!

➡ **Details in our paper!**

Takeaways

▲ Flexo

▲ TAE

✨ Flexo: fast and accurate weird gates with compiler support

✨ Transient Arch Exec: weird programs with powerful computational abilities

✨ µWMs are practical for program obfuscation or other attacks

- Bending microarchitectural weird machines towards practicality
- Transient architectural execution: from weird gates to weird programs
- Contact: Ping-Lun Wang, pinglunw@andrew.cmu.edu

Carnegie Mellon University CyLab Security and Privacy Institute

TEL AVIV UNIVERSITY אוניברסיטת תל אביב

RUHR UNIVERSITÄT BOCHUM RUB